

Toward SmartNIC-Based L7 Load Balancing with TOE and Pod Steering

Yuma Ueda
Keio University, AIST
Japan

Ryuichi Sakamoto
Institute of Science Tokyo
Japan

Takaaki Fukai
AIST
Japan

Takahiro Hirofuchi
AIST
Japan

Extended Abstract

Microservice has become a common approach to building large-scale applications in the cloud. A microservice application is split into fine-grained services, each deployed as an independent container, so network processing consumes a large share of the host’s CPU time. Because contention for CPU resources between network processing and microservices affects latency, especially tail latency, researchers have studied offloading network processing from the host CPU. Many functions, such as L4 load balancing, TCP processing, and packet delivery to containers, can be offloaded to hardware. What remains on the host is L7 load balancing. Our goal is to offload L7 load balancing and TCP processing to a SmartNIC.

Offloading the L7 load balancer to the SmartNIC must meet three requirements: (R1) the NIC terminates TCP and makes the L7 decision; (R2) it avoids load imbalance caused by connection-level steering; and (R3) it requires no changes to applications running on the host. Existing approaches fail to satisfy at least one requirement.

We propose an L7 load balancer that runs in the datapath of a SmartNIC with a TCP offload engine (TOE) and meets all three requirements. The SmartNIC carries a TOE that terminates client TCP connections, and the L7 load balancer runs on Arm cores. The TOE passes the reassembled TCP stream to the load balancer on the Arm cores, which parses HTTP/1.1 request boundaries and headers and selects a destination container for each request. The TOE then forwards each request to the PCIe function corresponding to its destination container. Between the SmartNIC and the containers, no additional TCP connections are established. A custom lightweight protocol identifies each connection with a single ID. Because client connections terminate on the TOE, changing the destination container per request requires no TCP state migration between containers. A custom socket emulation layer, inserted when the application in the container is loaded, lets applications use this protocol transparently. Container-to-container communication within a host can

use the existing in-kernel stack to avoid loading the Arm cores. R1 is met by colocating the TOE that terminates client TCP connections and the Arm cores that run the L7 load balancer on the SmartNIC. For R2, the Arm cores select a destination for each request, avoiding the restriction of assigning all requests in a connection to one container. For R3, the custom socket-emulation layer removes the need for application changes.

Our prototype extends ContaTOE [1] and runs on an Alveo U25. ContaTOE is a SmartNIC that integrates the Open-NIC Shell and a TOE: TCP terminates on the NIC, and the NIC steers each L4 connection to a physical function (PF) bound to a container via PCIe passthrough, while LD_PRELOAD-based socket-layer emulation allows unmodified applications to use the TOE. ContaTOE, however, provides no programmable core for an L7 load balancer, and its steering decision is fixed per connection. This work extends ContaTOE in two ways. First, we add Arm cores to the NIC and run the L7 load balancer on them under FreeRTOS. For each request within a connection, the load balancer can select a destination PF, so the request is delivered to the container bound to that PF. Second, we extend socket emulation from the libc symbol level to the system call level to enable future evaluation with more realistic workloads. Preliminary evaluation shows that our prototype reduces mean, median, and p99 latency by 23–62%, 20–32%, and 22–82%, respectively, over a host-side nginx reverse proxy across 4k–24k req/s.

Acknowledgments

This work was supported by JST CREST, Japan, Grant Number JPMJCR22M3.

References

- [1] Tatsuya Kawano, Ryuichi Sakamoto, Gongyu Deng, Takaaki Fukai, and Takahiro Hirofuchi. 2026. ContaTOE: FPGA SmartNIC-Based TCP Offloading for Scalable Multi-Container Networking. In *2026 IEEE 19th International Conference on Cloud Computing (CLOUD)*. 352–363. doi:10.1109/CLOUD72782.2026.00046